Smart Book System — Team Presentation

English Version | 20 Slides | Team ERB-Team-A

Slide 1: Title Slide

Smart Book System

A Digital Library Management Platform

Team ERB-Team-A Built with: Node.js, Express, MongoDB, and JavaScript **Presentation time:** 5 minutes per member

Slide 2: Team Leader — What This System Does

Smart Book System is a web application that helps a library manage its books and members online.

What users can do: - Browse the book catalog and search for titles or authors - Create an account and log in securely - Save books they are interested in - Borrow books and return them later - Read borrowed books directly in the browser - Manage the entire library as an administrator

Why it matters: The system replaces manual paper-based tracking with a modern, digital workflow that is available 24/7.

Slide 3: Team Leader — How the System Is Built

Frontend (what users see): - Simple web pages built with HTML, CSS, and JavaScript - No frameworks needed — just clean, responsive pages - Pages include the book gallery, login screen, member dashboard, admin console, and book reader

Backend (what runs the logic): - Node.js and Express handle requests from the browser - MongoDB stores all data: users, books, and borrow records - Passwords are encrypted before being saved - A secure token system keeps users logged in

Team workflow: The team leader set up the project structure, database connection, security rules, and deployment process.

Slide 4: Team Leader + Member 1 — Login and User Accounts

Page: `public/login.html` - A single page with two tabs: Login and Register - Users enter their name, email, and password - The system remembers logged-in users with a secure token - After login, users are sent to the right page based on their role

Backend logic: `controllers/authController.js` - Handles new user registration - Verifies email and password during login - Returns a secure token so the user stays logged in

Database: `models/User.js` - Stores each user's name, email, encrypted password, and role - Passwords are never saved as plain text - Roles control what each user can see and do

Slide 5: Team Leader + Member 4 — Admin Console and User Management

Page: `public/admin.html` - A dashboard where administrators can manage everything - Book management table with forms to add, edit, or remove books - User management table to create or remove member accounts - Borrow records table to see all library activity

Backend logic: `controllers/userController.js` and `controllers/bookController.js` - Lists all registered users - Creates new user accounts with the correct role - Removes users and cleans up their history - Manages the book inventory with validation

Database: `models/User.js` and `models/Book.js` - Users have a role that controls admin access - Books have a unique identifier and stock tracking

Slide 6: Team Leader + Member 5 — Book Gallery and Homepage

Page: `public/index.html` - The main page where anyone can browse available books - A search bar that filters books instantly as the user types - Each book shows its title, author, and availability status - Administrators see extra buttons to add or remove books

Backend logic: `controllers/bookController.js` - Returns the full list of books - Searches books by title or author - Provides book details and reading content

Database: `models/Book.js` - Stores book title, author, unique ID, total copies, and available copies - Tracks when each book was added or updated

Slide 7: Team Leader — Security and Access Control

How the system stays secure: - All sensitive pages require a valid login token - Only administrators can add, edit, or delete books and users - Passwords are encrypted and never exposed in responses - Every request is checked before reaching the protected data

User roles: - Regular members can browse, borrow, and return books - Administrators have full access to manage the library - The system automatically redirects users to the correct pages

Slide 8: Team Leader — Error Handling and API Structure

User experience focus: - If something goes wrong, the system shows a clear message instead of crashing - Form inputs are checked before being saved - Duplicate accounts or book IDs are rejected with helpful feedback

API structure: - `/api/auth` — handles login, registration, and profile - `/api/books` — handles browsing, searching, and managing books - `/api/borrow` — handles wishlists, checkout, and returns - `/api/users` — handles user management for admins - `/api/health` — confirms the system is running

Slide 9: Team Leader — Database and Demo Data

Database setup: `config/db.js` - Connects the application to MongoDB - Reads the connection address from a secure configuration file - Logs connection status for debugging

Demo data: `seed.js` - Pre-loads 12 sample books with titles, authors, and reading content - Creates a default administrator account for testing - Can be run safely multiple times without creating duplicates

Database models: - Users store account information - Books store inventory and content - Borrow records track the lifecycle of each loan

Slide 10: Team Leader — Development and Deployment

Team collaboration workflow: - Code lives on GitHub with protected main and development branches - Each feature is built on its own branch and merged via review - Team members follow a standard commit and push process

Getting started locally: 1. Clone the repository and install dependencies 2. Set up the environment configuration file 3. Run the database seed script 4. Start the server and open the browser

Going live: - Free hosting platforms like Render or Vercel can deploy directly from GitHub - A cloud MongoDB database stores data online - The system updates automatically when new code is pushed

Slide 11: Member 2 — Book and Inventory Module

Role: Member 2 owns everything related to the book catalog, search, stock tracking, and the reading experience.

What was built: - The main book gallery that users see on the homepage - Search functionality so users can find books quickly - Stock tracking that shows availability in real time - An admin interface for managing the book collection - A book reader page for reading borrowed books

Impact: This module is the core of the system because it handles the entire book lifecycle from creation to reading.

Slide 12: Member 2 — Book Gallery Page

Page: `public/index.html` - The first page users see when they visit the site - Shows all available books in a clean, card-based layout - Includes a search bar at the top for instant filtering - Books display their status: available, low stock, or out of stock - Administrators can add new books directly from this page

How it works: 1. When the page opens, it automatically loads all books from the server 2. As the user types in the search box, results update immediately 3. Each book card shows important information at a glance 4. Notifications confirm when an action succeeds or fails

Slide 13: Member 2 — Book Management Logic

Backend logic: `controllers/bookController.js` - Retrieves the complete book list for the gallery - Handles search requests by matching titles and authors - Creates new books with valid data and unique identifiers - Updates book details and stock levels safely - Removes books when they are no longer needed - Controls access to book content for the reader

Stock management: - New books start with full stock equal to the total quantity - Stock is adjusted automatically when books are borrowed or returned - Administrators can override stock levels within safe limits

Slide 14: Member 2 — Book Data Structure

Database: `models/Book.js` - Each book has a title, author, and unique identifier - The system tracks how many copies exist and how many are available - Books can store long-form content for the reader feature - Every book record includes timestamps for creation and updates

Data rules: - Every book must have a unique identifier to avoid duplicates - Available copies can never be negative - Available copies can never exceed the total quantity owned - Content is limited to keep performance smooth

Slide 15: Member 2 — Search, Stock Status, and Reader

Search experience: - Users type a few letters and see matching books instantly - The search looks at both the title and the author name - Results are sorted so the newest books appear first

Stock indicators: - Green badge means plenty of copies are available - Yellow badge warns that copies are running low - Red badge means the book is currently unavailable

Book reader: - A dedicated page displays book content in a clean format - Only users who have borrowed the book can access it - Administrators can read any book for review purposes - A back button returns users to their account dashboard

Slide 16: Member 3 — Borrow and Wishlist Module

Role: Member 3 owns the borrowing system, wishlist, checkout workflow, and the member account page.

What was built: - A wishlist feature so members can save books they like - A checkout request system for borrowing books - A state-based workflow that tracks each loan from request to return - A personal dashboard where members see their profile, wishlist, and borrow history - Automatic stock updates when books are borrowed or returned

Impact: This module brings the library to life by connecting users with books and managing the complete loan lifecycle.

Slide 17: Member 3 — Member Center Page

Page: `public/member.html` - A personal dashboard for every logged-in member - Shows the user's profile information at the top - Displays a wishlist of saved books with quick actions - Lists all current and past borrow records with clear status labels - Includes buttons to request books, confirm loans, and return books

How it works: 1. The page loads the user's profile and displays their name and email 2. It fetches the user's wishlist and borrow history from the server 3. Members can move a book from wishlist to checkout request 4. Members can return borrowed books with one click 5. Status badges make it easy to see where each book is in the process

Slide 18: Member 3 — Borrow and Wishlist Logic

Backend logic: `controllers/borrowController.js` - Adds books to a user's wishlist when they click "interested" - Converts a wishlist item into a checkout request - Approves or returns books while updating the inventory - Retrieves a user's complete borrowing history - Allows administrators to view all borrowing activity across the system

Borrowing workflow: - A book starts as "interested" when saved to the wishlist - It becomes "pending" when the user requests to borrow it - It changes to "borrowed" when the loan is approved - It ends as "returned" when the book comes back - Each transition updates the stock count automatically

Slide 19: Member 3 — Borrow Record Data Structure

Database: `models/BorrowRecord.js` - Each record connects a user to a specific book - The status field tracks where the book is in the borrowing process - Dates are recorded when a book is borrowed and when it is returned - The system prevents duplicate active records for the same user and book

Data rules: - A user cannot have multiple active requests for the same book - The status follows a strict path: interested, pending, borrowed, returned - Stock is never allowed to drop below zero - All changes are saved with timestamps for tracking

Slide 20: Member 3 — Integration, Testing, and Demo

How the pieces fit together: - The borrow system connects users and books through shared records - Stock levels in the book catalog update automatically during borrow and return - The member dashboard pulls data from multiple sources into one place - The frontend communicates with the backend using simple API calls

Testing highlights: - Register a new account and log in successfully - Browse books, save favorites, and submit a borrow request - Watch stock decrease when a book is borrowed and increase when returned - Verify that the reader page only works for books that have been borrowed - Check that administrators can manage all records from the console

Live demo: 1. Open the homepage and browse the book gallery 2. Log in as an administrator to manage the collection 3. Log in as a member to test the wishlist and borrow flow 4. Borrow a book and open it in the reader 5. Return the book and confirm the stock updates correctly

Presentation prepared for Team ERB-Team-A | Smart Book System Project